[Role, objective (why ?), AWS (1-5), Agentic AI (1-5)]

Amazon Bedrock || AgentCore || ?

[Deployment] → scaling

BREAKS (ET)

2 coffee breaks { AM — 10:30, PM — 3:00 } > 15 mins

Lunch Break — 12 PM (1 hr)

Yes = +1 , No = +2

Lab: Deploying Intelligent Agents with Amazon Bedrock AgentCore Runtime ✓ (1)

In this lab, you build AI agents that understand natural language, use custom tools, and coordinate with other agents to solve complex financial problems using Amazon Bedrock and Strands SDK. By the end, you will have deployed a production-ready multi-agent system using Amazon Bedrock AgentCore.

Lab: Integrate Agents with MCP Servers and Tools using AgentCore Gateway (4)

This lab teaches you to integrate an AI agent with external tools using Amazon Bedrock AgentCore Gateway and the Model Context Protocol (MCP). You create a Gateway with JWT authorization, register Lambda functions as MCP tool targets, connect a pre-deployed nutrition coach agent, and validate end-to-end tool invocations using natural language queries.

Lab: Build Context-Aware Agents with Amazon Bedrock AgentCore Memory (3)

This lab teaches how to build AI agents with persistent memory across sessions using Amazon Bedrock AgentCore Memory. The scenario is an internal IT help desk agent that remembers employee device information and preferences across multiple sessions, showing firsthand how memory transforms a stateless agent into a context-aware assistant.

Lab: Secure AI Agent Access with Amazon Bedrock AgentCore Identity ✓ (2)

This lab teaches you to deploy an agent to Amazon Bedrock AgentCore Runtime with AgentCore Identity inbound auth.

* Agentic AI ?

✓ Autonomous

✓ FM/LLM ⇒ limitation → "Context window"

✓ FM / LLM ⇒ Limitation → Context window

Memory
- Short-term → current session / conversation
- Long-term → semantic, preferences, summary

AgentCore Memory

✓ Multi-agent system
↳ MCP & A2A

AgentCore Gateway → Tools, MCP

✓ Identity → Inbound Auth → who can get access to the Agent?
→ Outbound Auth → what Agents can get access to?

AgentCore Identity

✓ Open Source Frameworks
1. Langchain, LangGraph
2. Strands Agent ★
3. CrewAI ✓ Role Based agents
4. LlamaIndex ✓ Data Centric App.

Integration with Amazon Bedrock APIs

[Amazon Bedrock AgentCore Runtime
Frameworks, Models, Tools - - -]

Agents ⇒ Goal oriented

Amazon Bedrock AgentCore

↳ A collection of managed services for

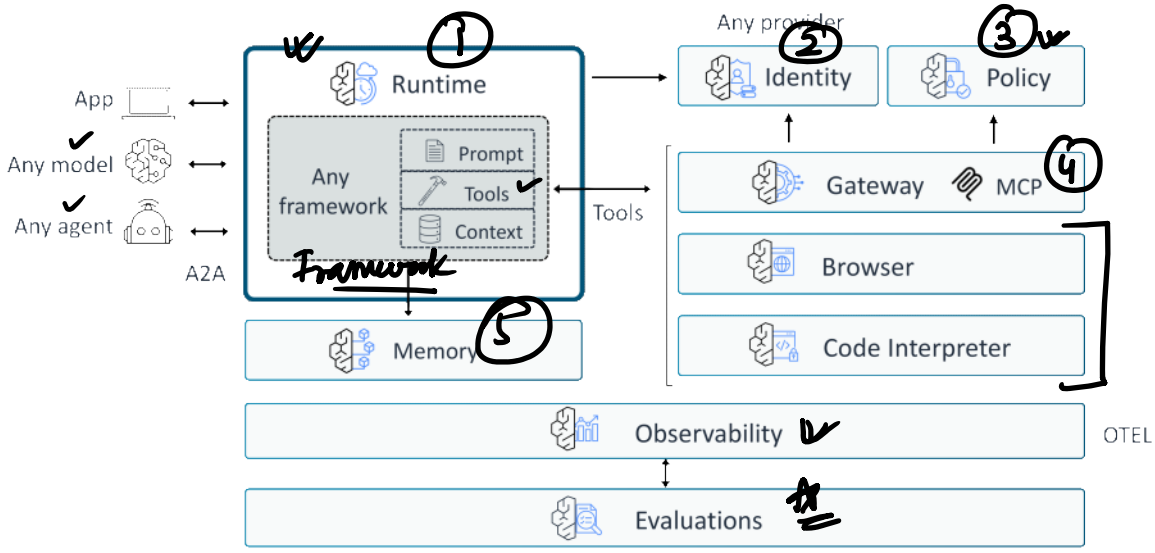
↳ Provides a collection of managed services for Building & operating production AI Agents

↳ Designed to support different Agent frameworks & models.

1. AgentCore Runtime ⇒ Execution environment for the Agents
2. AgentCore Identity ⇒ Identity & Authentication (Who? What?)
3. AgentCore Gateway ⇒ Standardized access to external tools & APIs
4. AgentCore Memory ⇒ Context Management
5. AgentCore Browser ⇒ Managed Browser-Based interaction
6. AgentCore Code Interpreter → Isolated code execution
7. AgentCore Policy ⇒ Authorization decision for protected Tools call.
8. AgentCore Observability = visibility into agent execution (Monitoring) (watching)

& AgentCore Evaluations = Access the quality of agent behavior)

AgentCore Architecture



Trad App Development

Code → EC2 / ECS / Lambda.

Agent → invoke → Tools
→ Perform → reasoning cycle
→ Maintain state

AgentCore Runtime → Managed Infrastructure

1. Framework & Model flexibility.
2. Protocol support → MCP & A2A
   - MCP (Model Context Protocol.
     - Interact with tools & resources
   - A2A (Agent to Agent)

User → App → Agent → Model, Tools

Lab 1 - Task 3

AgentCore → Runtime
→ Observability

v

④ observability

→ Identity

③ Memory → Context Management ✓

1- Memory & ~~Gateway~~ Identity

AgentCore Runtime

↳ supports Background processing.

AgentCore Python SDK

↳ Task management support

→ app.add_async_task() (A)

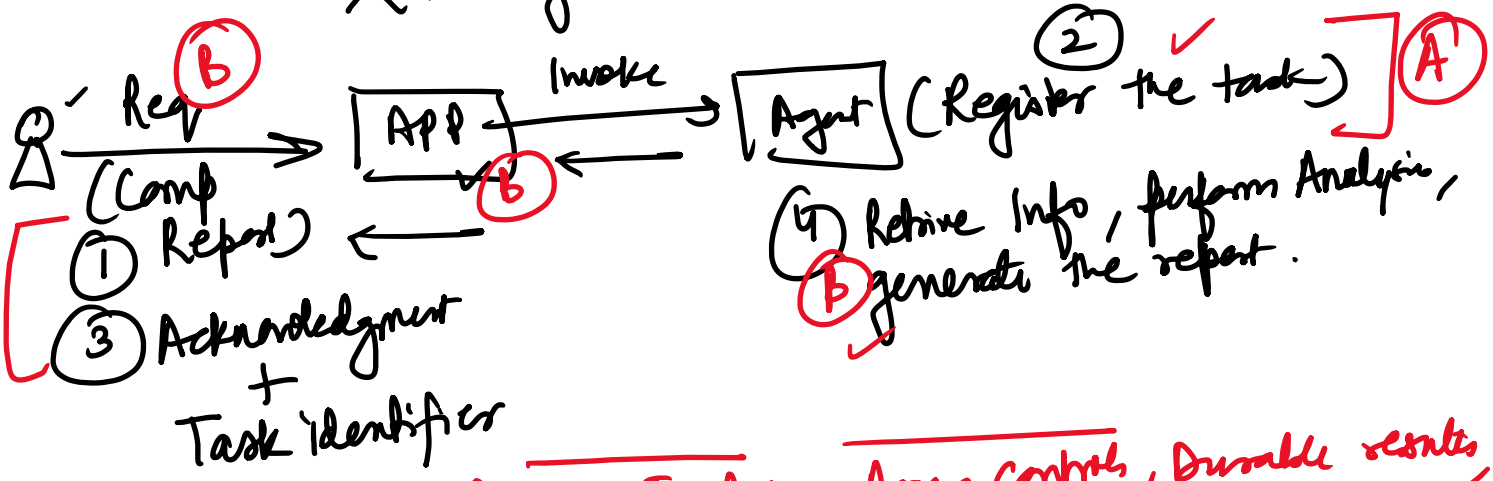
→ app.complete_async_task() (B)

why this is IMP?

→ Application is actively processing Background work.

App → Active Background work ⇒ HealthyBusy.

X Background tasks.

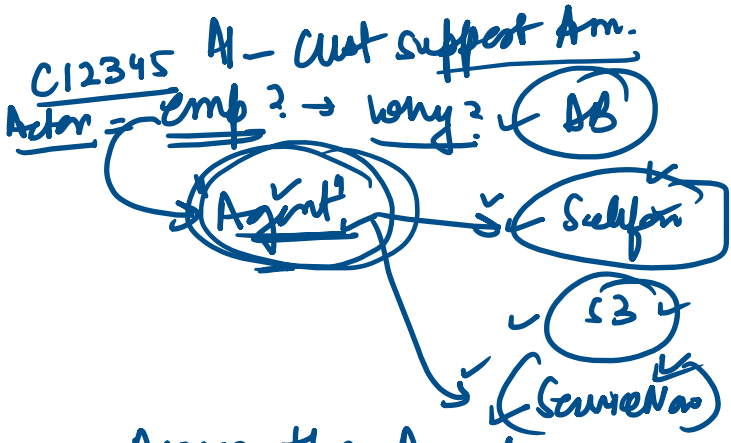


Error handling, ... potentially retry/recovery.

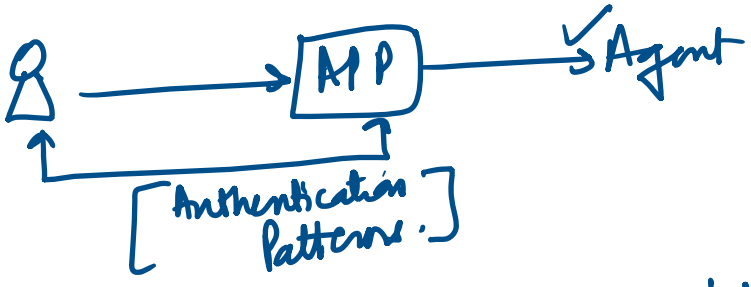
Agent Core Identity:

- Inbound
- outbound

- ✓ IAM
- ✓ OAuth



① Inbound ✓ Auth. → who can Accen the Agent?



- ✓ Direct user Authentication ⇒ Identity Provider * ① (Amazon Cognito) ✓
- ✓ Enterprise federation ⇒ Microsoft Entra ID / Okta ②
- ✓ System-to-system ⇒ API keys at an application or API Gateway ③

→ Amazon Bedrock [AgentCore] "Runtime" — Two inbound Auth

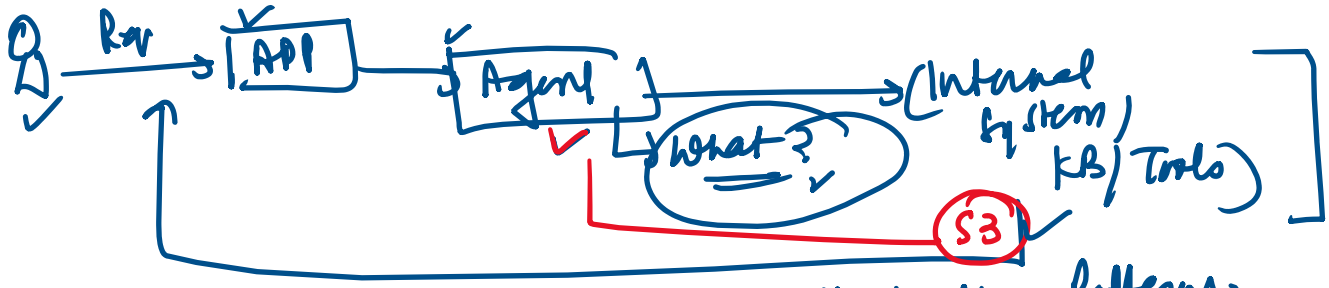
① IAM "SigV4" → signs API request using AWS credentials
↓
Agentcore Runtime
↓
"AWS-native workloads"

② ✓ JWT (JSON web Token) bearer-token Auth
↳ Application Authenticates users through external identity Provider (Amazon Cognito)

who? what? ... same.

(Authentication (who?) & Authorization (what?)) → NOT the same.

✓ valid JWT → caller has been Authenticated.



Outbound Authentication & Authorization Patterns:

1. AWS IAM & SigV4. (AWS Resources)

Agent → Running → AgentCore Runtime (IAM execution Role) → workload / AWS Service.

S3 → S3 Bucket → IAM + Policy → objects (Read ONLY)

2. OAuth (3rd Party — Salesforce, ServiceNow, Google --)

2LO (Two-legged Authentication)	3LO (Three legged)
Application Authenticates as itself for 'machine to machine' operations.	When access must be delegated by an individual user.

AgentCore "Memory"

Agent Core Memory

Q Why do AI Agents need memory?

Ans: Agentic Memory → Benefits

1. ✓ Contextual Intelligence.
2. User preferences ✓
3. Knowledge retention ✓

(1) SHORT-term & LONG-term Memory

Amazon Bedrock Agent

[SHORT-term] ⇒ Raw Information associated with a conversation

- → user messages, assistant responses, tool interactions, session attributes, agent checkpoints
- → current conversation, session details

LONG-TERM Memory = useful info extracted from interaction.

- → Extracting, consolidating, organizing important Information from PRIOR interactions.

✓ SHORT-term ⇒ Tells the agent what happened in conversation

✓ Long term ⇒ Tells the agent what useful Info it should remember

U should remember

Long-Term → Memory strategies

1. user preference memory
2. Semantic memory ⇒ facts, concepts, Relationships
   - ORD → Postgre SQL
   - App deployed - us-east
   - Dev → Python
3. Summary

Implementing AgentCore Memory

1. Memory Resource

AgentCore SDK for Python

└ import MemoryClient from bedrock_agentcore.memory

→ create_memory

└ name, descrip, config options

↓

event_expiry_days

2. [Events, actors, sessions]

a) Event = [Interaction]

└ user msg, assist resp, tool-info

b) Actor = whose memory we're dealing with

Memory ID ↓ Event

c) Session group → Interaction belong to a particular conversation

[Actor ID = Customer A

└ session ID = Session 001 (3 Req/Qs, 3 Resp/Ans)

3 Major operations:

1. CreateEvent
   - memoryID, actorID, sessionID & messages.
2. ListEvents:
   - GetEvent = individual event using eventID

Long-term = namespaces

1. Summary = event (actorid + sessionid)
2. Semantic = actorid
3. Preference = actorid
4. Episodes = actorid + session id

AgentCore Gateway

Q What is MCP (Model Context Protocol)?

- 10 AI Agents + 5 Agent
- 20 Enterprise system — Salesforce, Service Now, Internal DB, payment — —

10 system.

THREE Components of MCP:

1. MCP Host → AI App — env in which MCP connections
2. MCP Client → within Host — establish & managing comm with MCP Server
3. MCP Server → Expose capabilities → tools.

③ MCP server → Expose capabilities → tools.

(1) Tools ✓
(2) Resources.
(3) Prompt

Amazon Bedrock AgentCore Gateway

↳ Managed Connectivity layer through which AI Agents can access external tools & services

Core Capabilities of AgentCore Gateway

(1) Unified Tool Access

Agent → Gateway Endpoint → Discover Tool

(2) Authentication HANDING

✓ Gateway supports authenticated inbound requests & configure outbound connections to Backend.

(3) MCP Protocol support.

↳ Agents can discover & invoke tools → MCP operations

(4) Multiple Target Types:

↳ Lambda, REST APIs, Supported MCP servers.

Gateway Architectural Components

(1) Gateway
1. Managed MCP endpoint through which Agents

(1) Gateway
↳ Managed MCP endpoint through which Agents communicate

(2) Target
↳ Targets connect Gateway to Backend services

(3) Authorizer = Authenticate incoming Reqs.

(4) Service Role = IAM Role that gives Gateway the permissions to access AWS services